山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 8 散列表		
实验学时：2	实验日期： 2023.11.8	

实验目的：

1.使用线性开型寻址实现

2.使用链表散列方式

软件开发工具：

DEV C++

1. 实验内容

实验 A：线性开型寻址

内容：

给定散列函数的除数 D 和操作数 m，输出每次操作后的状态。

有以下三种操作：

插入 x，若散列表已存在 x，输出“Existed”，否则插入 x 到散列表中，输出所在的下标。

查询 x，若散列表不含有 x，输出“-1”，否则输出 x 对应下标。

删除 x，若散列表不含有 x，输出“Not Found”，否则输出删除 x 过程中移动元素的个数。

实验 B：链表散列

内容：

给定散列函数的除数 D 和操作数 m，输出每次操作后的状态。

有以下三种操作：

插入 x，若散列表已存在 x，输出"Existed"

查询 x，若散列表不含有 x，输出"Not Found"，否则输出 x 所在的链表长度

删除 x，若散列表不含有 x，输出"Delete Failed"，否则输出 x 所在链表删除 x 后的长度

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

所需要的数据结构与算法：散列表，哈希函数映射，线性表，链表

整体思路描述：

对于实验 A：线性开型寻址

实验 A 的难点在于实现删除操作时，不能简单将所在元素删去，而应当继续探查删除元素后方元素，判断其“初始桶是否在删除操作造成空桶的前方”，如果满足则应当将该元素移动到空桶处，并在原来位置形成一个空桶；如果不满足则不移动，并持续向数组后方探查，直到遇到一个空桶时停止探查。判断“初始桶是否在删除操作造成空桶的前方”有三种可能性：

可能性 1 初始桶在空桶前方，并且空桶在遍历元素前方

可能性 2 空桶在所遍历元素前方并且遍历元素在初始桶前方

可能性 3 遍历元素在初始桶前方并且初始桶在空桶前方

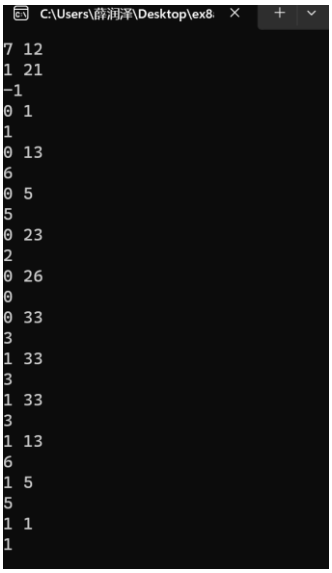
满足上述三种可能性则将遍历到的元素移动到空桶处，直到遍历到一个空桶结束。

对于实验 B：链表散列

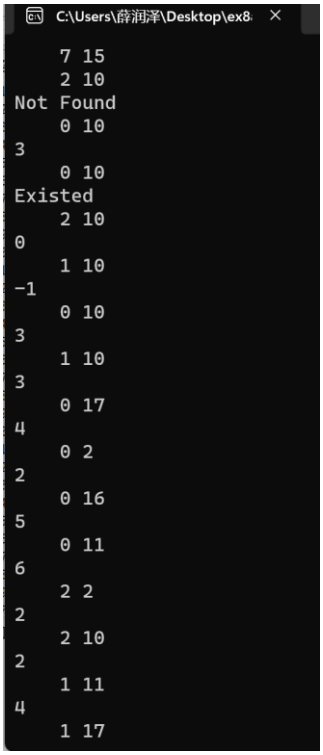
实验 B 使用链表来构建散列表，当一个元素的 key 值模 D 与其他元素 key 值模 D 相等时，在线性表中使用线性探查的方式将冲突元素放入数组后方空位；在链表中，则可以将有冲突的元素的指针连接到上一个元素的 next。

3. 测试结果（测试输入，测试输出）

实验 A：线性开型寻址



实验 B：链表散列



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

问题分析与解决：

在写线性开型寻址时，我初步理解“遍历到的元素的初始桶在空桶的前方”是其初始桶在空桶的左边或者正好是该空桶，忽略了遍历元素从数组后方循环到前方的两种情况。通过添加逻辑，当遍历元素索引小于初始桶索引，并且空桶索引小于遍历元素索引以及当遍历元素索引小于初始桶索引，并且空桶索引大于初始桶索引等两句判断语句，成功实现了散列表的删除操作。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

实验 A：线性开型寻址

```
#include<iostream>
```

```
class hash{
public:
    hash(int D){
        element = new int[D];
        sym = new bool[D];
        for(int i = 0; i<D; i++){
            sym[i] = 0;
        }
        arrayLength = D;
    }
    ~hash(){
        delete []element;
        delete []sym;
    }
    void insert(int x){
        int mapnum = x % arrayLength;
        int countnum = 0;//判断是否循环过一遍
        while(countnum < arrayLength && sym[mapnum] == 1){
            if(element[mapnum] == x){
                std::cout<<"Existed"<<'\n';
                return ;
            }
            countnum++;
            mapnum++; mapnum = mapnum % arrayLength;
        }//直到循环过一遍或者找到了一个空位停止
        if(sym[mapnum] == 0){
            element[mapnum] = x;
            sym[mapnum] = 1;
            std::cout<<mapnum<<'\n';
        }
        return;
    }

    void search(int x){
        int mapnum = x % arrayLength;
        int countnum = 0;//用来判断循环是否循环过一遍
        while(countnum < arrayLength){
            if(sym[mapnum] == 0){
                std::cout<<"-1"<<'\n';
                return;
            }
            if(element[mapnum] == x){
                std::cout<<mapnum<<'\n';
                return;
            }
            countnum++; mapnum++;
```

```
            mapnum = mapnum % arrayLength;
        }
        std::cout<<"-1"<<'\n';//遍历了整个数组都没有找到
        return;
    }
    //遍历到位置的元素索引和初始桶做减法，如果>0，要求空位在初始桶后，<0 空位在初始桶前
    void deleteelement(int x){
        int mapnum = x % arrayLength;
        int countnum = 0;//用来判断循环是否循环过一遍
        while(countnum < arrayLength && sym[mapnum] == 1){
            if(element[mapnum] == x){
                break;
            }
            countnum++; mapnum++;
            mapnum = mapnum % arrayLength;
        }
        if(countnum == arrayLength || sym[mapnum]==0){
            std::cout<<"Not Found"<<'\n';
            return;
        }
        //剩下的情况是找到且 mapnum 就是 x 对应的索引
        //进行移动操作
        int count = 0;//记录移动个数
        sym[mapnum] = 0;//删除该位置元素，即将该位置的 sym 置为 0
        int last = mapnum;//用来记录空缺桶位置
        mapnum++; mapnum = mapnum % arrayLength;
        countnum =   0;//统计是否完成表遍历
        while(countnum < arrayLength && sym[mapnum] == 1){//没有遇到空桶且没有完成一次遍历
            int start = element[mapnum] % arrayLength;
            //如果 1  初始桶在空桶前方，并且空桶在遍历元素前方
            //如果 3  空桶在所遍历元素前方并且遍历元素在初始桶前方
            //如果 2  遍历元素在初始桶前方并且初始桶在空桶前方
            //mapnum 遍历位置，  startbucket 初始桶，  last  空桶
            if((start <= last && last < mapnum) || (last < mapnum && mapnum < start) || (mapnum < start
&& start <= last)){//如果该元素的初始桶位置在空缺位置左边
                element[last] = element[mapnum];
                sym[last] = 1;
                sym[mapnum] = 0;
                last = mapnum;
                mapnum++; mapnum = mapnum % arrayLength;
                count++;
            }
            else{
                mapnum++; mapnum = mapnum % arrayLength;
            }
        }
```

```
            std::cout<<count<<'\n';
            return;
        }
        void Outhash(){
            for(int i = 0; i < arrayLength; i++){
                if(sym[i]== 1)
                std::cout<<element[i]<<' ';
                else
                std::cout<<'-'<<' ';
            }
            std::cout<<'\n';
        }

private:
        int* element;//用来记录表
        bool * sym;//用来记录桶是否空,0 为空， 1 为占用
        int arrayLength;//记录哈希表的表长度,用来取模值
};
int main(){
        int D, m;
        std::cin>>D>>m;
        hash h(D);
        int opt; int x;
        for(int i = 1; i<=m; i++){
            std::cin>>opt>>x;
            if(opt == 0) h.insert(x);
            else if(opt == 1) h.search(x);
            else if(opt == 2) {
            h.deleteelement(x);
            }
        }
        return 0;
}
```

实验 B:链表散列

```
#include<iostream>
struct Node{
        Node * next;//连接下一个节点的指针
        int k;//key 值
};

class hash{
public:
        hash(int D){
            arrayLength = D;
```

```
        element = new Node* [D];//element 是 Node*类型的数组
        for(int i = 0; i < D; i++){
            element[i] = nullptr;//把数组每一个指针元素置为空指针
        }
    }
    ~hash(){
        for(int i = 0; i < arrayLength; i++){
            Node* ptr = element[i];
            while(ptr != nullptr){//将所有的 new 出的 Node 节点全部 delete
                Node* temp = ptr->next;
                delete ptr;
                ptr = temp;
            }
        }
        delete []element;
    }
    void insert(int x){
        int mapnum = x % arrayLength;
        if(element[mapnum] == nullptr){
            element[mapnum] = new Node;
            element[mapnum]->k = x;
            element[mapnum]->next = nullptr;
            return;
        }
        Node* cntptr = element[mapnum];//记录遍历到的节点
        Node* leftptr = nullptr;//记录上一个节点
        while(cntptr != nullptr){
            if(cntptr->k == x){
                std::cout<<"Existed"<<'\n';
                return;
            }
            leftptr = cntptr;
            cntptr = cntptr->next;
        }//直到整个 cnt 到链表末尾结束
        //没有找到 x，则在链表末尾插入 x
        Node* temp = new Node;
        temp->k = x;
        temp->next = nullptr;
        leftptr->next = temp;
        return;//
    }
    void search(int x){
        int mapnum = x % arrayLength;
        if(element[mapnum] == nullptr){//如果链表中第一个元素为空，则不存在 x 元素
            std::cout<<"Not Found"<<'\n';
            return;
```

```
        }
        int count = 0;
        Node* tempptr = element[mapnum];
        int sym = 0;//探查是否找到
        while(tempptr != nullptr){
            count++;
            if(tempptr->k == x){
                sym = 1;
            }
            tempptr = tempptr->next;
        }
        if(sym == 1)std::cout<<count<<'\n';
        else std::cout<<"Not Found"<<'\n';
        return;
    }
    void hashdelete(int x){
        int mapnum = x % arrayLength;
        if(element[mapnum] == nullptr){//如果链表第一个元素为空，则不存在 x 元素
            std::cout<<"Delete Failed"<<'\n';
            return;
        }
        int count = 0;//记录链表删除 x 后的长度
        if(element[mapnum]->k == x){//如果要删除的节点是头结点
            Node* temp = element[mapnum]->next;
            delete element[mapnum];
            element[mapnum] = temp;
        }
        Node* cntptr = element[mapnum];//记录遍历到的现在的元素指针
        Node* leftptr = nullptr;//记录上一个元素的指针
        while(cntptr != nullptr){//当没有遍历到链表尾部
            if(cntptr->k == x){//如果找到该删除元素，则将其删除
                Node* temp = cntptr->next;
                delete cntptr;
                leftptr->next = temp;
                cntptr = temp;
            }
            else{                   //如果不是删除元素，则将 count++，并将两个指针移动到下一个
                count++;
                leftptr = cntptr;
                cntptr = cntptr->next;
            }
        }
        std::cout<<count<<'\n';
        return;
    }
    void Outhash(){//用于输出整个哈希表
```

```
            for(int i = 0; i<arrayLength; i++){
                Node* temp = element[i];
                while(temp != nullptr){
                    std::cout<<temp->k<<' ';
                    temp = temp->next;
                }
                std::cout<<'\n';
            }
            return;
        }


private:
        Node **element;//定义一个指针数组
        int arrayLength;//存储 D 的大小
};
int main(){
        int D, m;
        std::cin>>D>>m;
        hash h(D);
        for(int i = 1; i <= m; i++){
            int opt, x;
            std::cin>>opt>>x;
            if(opt == 0){
                h.insert(x);
            }
            else if(opt == 1){
                h.search(x);
            }
            else if(opt == 2){
                h.hashdelete(x);
            }
        }
        return 0;
}
```